Week 9 · Making it look right: visual design & accessibility

Technical Interaction Design · ITU · Autumn 2026

Built 19 October 2026 · commit `b29342f`
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-09.pdf

CONTENTS

Libraries

- Most of the code your app runs, you did not write
- `npm` reads `package.json` and fetches everything it lists
- The libraries land in `node_modules`, which never goes into Git
- `npm explain` says why a package is in `node_modules`
- A version number says how much changed: major, minor, patch
- `^` lets npm take newer minor and patch versions, but not a new major one
- `package-lock.json` pins the whole tree, and you commit it
- A new major version is a reading assignment
- Every dependency is a decision someone has to defend
- `npm install` runs strangers' code on your laptop, and later in your users' browsers
- `npm audit` lists known vulnerabilities, and somebody has to read the list
- Every dependency ships to your users' browsers
- A license says what you may do with the code
- Let one file import a library, and leaving it stays cheap
- You call a library; a framework calls you

Libraries

Source: Lectures/Technical/Tooling/Libraries.md — something unclear? Open an issue, or send a pull request.

“If I have seen further, it is by standing on the shoulders of giants.” (Isaac Newton)

(the programmers’ version of the same idea)

Most of the code your app runs, you did not write

Look at the `dependencies` in `todo-26`’s `package.json`:

```
"dependencies": {
  "events": "^3.3.0",
  "parse": "^8.6.0",
  "react": "^19.2.8",
  "react-dom": "^19.2.8",
  "react-router-dom": "^7.18.4"
}
```

React itself is on that list. You did not write the component model, the router or the Parse SDK, and you should not. Every modern language lets a project declare the libraries it depends on, in one file, and JavaScript’s is `package.json`.

Something unclear in this section? Open an issue.

`npm` reads `package.json` and fetches everything it lists

Each language has its own **package manager**; JavaScript’s is `npm`. It runs on **Node**, which is JavaScript outside the browser, built on the same engine as Chrome (V8). Node is also what runs Vite, the dev server, and the build. Your React app itself still runs in the browser; Node is the workshop it is built in.

```
npm install                    # everything package.json lists
npm install @mui/material      # add one more, and write it into package.json
```

Run `npm install` after you clone a project, and again after every `git pull` that changed `package.json`. If the app suddenly cannot find a package a teammate just added, that is why.

Something unclear in this section? Open an issue.

The libraries land in `node_modules`, which never goes into Git

`npm install` puts the code of every library, and of every library *they* depend on, into `node_modules/`. In `todo-26` that is 217 packages and about 115 megabytes. Only 16 of them come from the five lines in `dependencies`; the rest are the tools in `devDependencies` (Vite, ESLint, `gh-pages`) and what *they* depend on. Those run on your laptop and never reach the user's browser.

None of it belongs in the repo. Anybody can recreate it with `npm install`, and some of it is built for one particular operating system. That is why `node_modules` is in `.gitignore`.

Something unclear in this section? Open an issue.

`npm explain` says why a package is in `node_modules`

Nobody in the group installed `ws`, but it is in `node_modules`. `npm explain` says who did:

```
$ npm explain ws
ws@8.20.0
node_modules/ws
  ws@"8.20.0" from parse@8.6.0
  node_modules/parse
    parse@"^8.6.0" from the root project
```

Parse brought it, for its live queries. `npm ls --all` prints the whole tree at once.

Something unclear in this section? Open an issue.

A version number says how much changed: major, minor, patch

Most JavaScript libraries use semantic versioning. In `19.2.8`:

- **19**, the *major* version, changes when something that used to work stops working
- **2**, the *minor* version, changes when something new is added, and the old still works
- **8**, the *patch* version, changes for bug fixes only

Not everybody follows it. Some projects use the date (*calendar versioning*: `pip` 24.0 came out in 2024). TeX's version number gets one more digit of π with every release.

Something unclear in this section? Open an issue.

`^` lets npm take newer minor and patch versions, but not a new major one

`"react": "^19.2.8"` means *19.2.8 or anything newer that is still 19*. That is a **flexible** version: you get bug fixes without asking. `"react": "19.2.8"`, with no `^`, is **pinned**: exactly that version, and nothing else.

But even pinning everything in `package.json` does not pin what your libraries depend on. `parse` has dependencies of its own, which have their own, and most of their versions are flexible too.

Something unclear in this section? Open an issue.

`package-lock.json` pins the whole tree, and you commit it

So `npm` writes down every version it actually installed, for every package in `node_modules`, in `package-lock.json`. The next `npm install`, on any laptop, installs exactly those.

That is the point of it, and why it goes into Git, even though `npm` generates it. Without it, two people in the same group, who ran `npm install` a week apart, can have different versions of the same library, and a bug that only one of them sees. `todo-26` commits it.

Do not edit it by hand. When two branches both added a package and it conflicts on merge:

1. Resolve `package.json` by hand, and keep both packages. It is short, and it is the file that says what you want.
2. Take either branch's `package-lock.json` whole, for example with `git checkout --theirs package-lock.json`. It is now missing the other branch's package.
3. Run `npm install`. It sees the package that `package.json` asks for and the lock file lacks, installs it, and writes it into the lock file.
4. `git add` both files and finish the merge.

You only ever resolve `package.json`; `npm` regenerates the rest from it.

Something unclear in this section? Open an issue.

A new major version is a reading assignment

`^` keeps a new major version from arriving by accident. It does not keep you on the old one forever: `npm outdated` lists every package that has a newer version, and sooner or later the one you are on stops getting fixes.

A new major version means something stopped working, and the library says what in its changelog or migration guide. Read it before you upgrade. React Router 7, for example, merged `react-router-dom` into `react-router`: the upgrade guide tells you to import from `react-router` instead. `todo-26` still imports from `react-router-dom`, which keeps working because version 7 still publishes it, as a copy of `react-router`.

GitHub's Dependabot can open a pull request for each new version. In Git Flow it is then reviewed like any other pull request.

Something unclear in this section? Open an issue.

Every dependency is a decision someone has to defend

Adding a package takes one command. Getting rid of it, once forty files import it, takes a week. So before you `npm install` something, ask:

- **Do we need it?** Formatting one date does not need a date library.
- **Is it alive?** When was the last release? Does anybody answer its issues?
- **Can we trust it?** Is it the package you think it is, and who can publish a new version of it?
- **What does it cost?** How much bigger does the app get, and what do you give up by working its way instead of yours?
- **May we use it?** What does its license ask of us?

`react-router-dom` earned its place in week 6: writing your own router is a project of its own. MUI, next, is a bigger decision: a whole design language, in exchange for a type scale, a palette and spacing that already work together. It is the *No needless dependencies* item on the review checklist.

Something unclear in this section? Open an issue.

`npm install` runs strangers' code on your laptop, and later in your users' browsers

Every package is code written by people you have never met, and every new version of it is published by whoever holds the keys at the time. Three stories:

- **left-pad, 2016.** After a dispute over a package name, a developer unpublished all of his packages. One of them, `left-pad`, was 11 lines that pad a string, and thousands of projects depended on it without knowing, Babel among them. Builds broke around the world for an afternoon. Since then, npm no longer lets you unpublish a package that others depend on.
- **event-stream, 2018.** The maintainer of a package with two million weekly downloads, tired of it, handed it to a volunteer. The volunteer added a dependency whose code stole bitcoin from users of one particular wallet app.
- **chalk and debug, 2025.** A maintainer of some of npm's most used packages got a phishing email that looked like it came from npm. With his account, the attackers published versions that, in the browser, swapped the address in crypto payments for their own. The packages had more than two billion weekly downloads together; the versions were caught within hours. A week later came Shai-Hulud, a worm that, on install, stole the developer's npm and GitHub tokens and used them to publish itself into that developer's other packages.

Installing can run code, too: a package can declare an install script, which `npm` runs on your machine. And some attacks just wait for a typo: in 2017, `crossenv` imitated the popular `cross-env` and sent your environment variables, passwords included, to its author.

The lock file is part of the defence: you get the version you already tried, not the one published last night. So are reading a package name twice before you install it, and having fewer packages in the first place.

Something unclear in this section? Open an issue.

`npm audit` lists known vulnerabilities, and somebody has to read the list

`npm audit` checks every installed version against a database of reported vulnerabilities. In October 2026, for `todo-26`, it reports nine, eight of them *high*.

Most are in `braces` and `brace-expansion`, which pattern-matching tools use, here through `gh-pages`: a carefully crafted pattern can make them run for a very long time. That matters for a server that matches patterns users send it, and not at all for a deploy script on your laptop. The suggested `npm audit fix --force` would "fix" it by installing an *older* `gh-pages`, a breaking change.

`npm audit --omit=dev` leaves out the tools and looks only at what the app uses. It reports one: Parse 8.6.0 pins `ws` to exactly 8.20.0, which has two known problems. Parse 8.6.2 uses a fixed `ws`, and `^8.6.0` already allows it, so `npm audit fix` updates Parse and the warning is gone.

So the count alone says little. Read each one and ask: does this code run in our app, and can a user reach it?

Something unclear in this section? Open an issue.

Every dependency ships to your users' browsers

`npm run build` prints how big the app is. For `todo-26`:

	size	gzipped	packages shipped
before	507 kB	155 kB	16
MUI's `TextField` in `TextInput`, and its `Button` in `AddForm`	694 kB	214 kB	102

Two components made the app almost 40% bigger. That might be a good trade, but it is a trade, and every user pays it on every first visit. bundlephobia.com shows the size of a package before you install it.

Sometimes the browser already has what you need, for 0 kB:

```
new Intl.DateTimeFormat("da-DK", { dateStyle: "long" }).format(date);
// "29. oktober 2026"

new Intl.RelativeTimeFormat("en", { numeric: "auto" }).format(-1, "day");
// "yesterday"
```

Something unclear in this section? Open an issue.

A license says what you may do with the code

Every package on npm states a license, on its npm page and in its own `package.json`. Most of what you use is under one of two kinds:

- **Permissive** licenses, like MIT (React, React Router, MUI) and Apache 2.0 (Parse): do what you like, including selling it, as long as you keep the copyright notice.
- **Copyleft** licenses, like the GPL: if you give somebody software that includes it, you have to give them your source code too, under the same license. JavaScript sent to a browser counts as given.

For a course project this rarely matters. For a company it can decide whether a library is allowed at all.

Something unclear in this section? Open an issue.

Let one file import a library, and leaving it stays cheap

If forty files import a library, forty files change when you replace it. If one file imports it, and the rest of the app imports that file, one file changes.

`todo-26` already does this twice. `TextInput.jsx` wraps the browser's `<input>`: the forms use `TextInput`, so changing what it renders changes every form at once. And the service layer wraps Parse: components call `addTodo(list, text)`, and only `services/` knows that Parse exists.

A new package gets the same treatment. `todo-26` reports errors with `alert()`. To show them as toasts with `react-hot-toast`, write one module:

```
// notify.js: the only file that knows which toast library we use
import toast from "react-hot-toast";

export function notifyError(message) {
  toast.error(message);
}
```

and every component calls `notifyError(error.message)`. If the library dies, or you find a better one, you rewrite `notify.js` and nothing else.

You cannot do this with everything. React is in every component file, and there is no wrapping it. The next section says why.

Something unclear in this section? Open an issue.

You call a library; a framework calls you

In `notify.js`, our code calls `toast.error` when *it* decides to. React works the other way round. You write `App` and the components, but you never call `App()`: React does, whenever it decides the screen must be drawn again. Code that calls yours is a **framework**.

React calls itself a library, and next to frameworks like Angular it is a small one. But for your components it behaves like a framework: every one of them is written in its shape, with props in, JSX out, and hooks used by its rules. That is why it cannot be wrapped, and why leaving Parse means rewriting `services/`, while leaving React means rewriting the app.

Something unclear in this section? Open an issue.

Exam Questions

1. How do you install a dependency with `npm`, and what do you do to make sure that your teammates use the same version you do?

2. When do you have to run `npm install`?

3. What is the `node_modules` folder for? Should you add it to version control? Why or why not?

4. What do the three numbers in a version like `19.2.8` mean, and what does the `^` in front of it allow?

5. Why do we need `package-lock.json`, and do we commit it?

6. Two branches each added a different package, and `package-lock.json` conflicts on merge. How do you resolve it?

7. Why would you let only one file in your app import a given library? Give an example from `todo-26`.

8. Your `node_modules` contains a package nobody in the group installed. How do you find out why it is there?

9. Name two ways an npm package can harm you or your users, and one thing you can do about each.

10. `npm audit` reports eight high vulnerabilities. Do you run `npm audit fix --force`? What do you check first?

11. How do you find out how much a library adds to the size of your app, before and after you install it?

12. What is the difference between a library and a framework? Which of the two does React behave like, and what does that mean if you ever want to leave it?